

TP n°1 : Découvrir OMNeT++

Objectif du TP

- Se familiariser avec le simulateur **OMNeT++**
- Apprendre les bases de la **conception, implémentation et simulation** d'un **réseau de capteurs sans fil**

Présentation de OMNeT++

- **OMNeT++** est une plateforme de simulation :
 - Modulaire
 - Orientée objet
 - À événements discrets
- Écrite en **C++**
- **Open source**, utilisée à des fins académiques
- Intègre :
 - Un IDE basé sur **Eclipse**
 - Un environnement graphique de simulation

Domaines d'utilisation

OMNeT++ permet de modéliser et simuler :

- Réseaux filaires et sans fil
 - Protocoles de communication
 - Réseaux de files d'attente
 - Systèmes distribués et multiprocesseurs
 - Systèmes logiciels complexes
- Plus généralement : tout système à événements discrets basé sur l'échange de messages

Architecture d'OMNeT++

- Organisation **hiérarchique** de modules

Types de modules

- **Modules simples**
 - Implémentés en C++
 - Chaque module → fichiers **.cc** et **.h**
- **Modules composés**
 - Regroupement de modules simples ou composés
 - Nombre de niveaux hiérarchiques illimité
- **Module système (Network)**
 - Niveau le plus élevé
 - Créé par l'utilisateur

Communication entre modules

- Basée sur l'échange de messages
- Les modules simples :
 - Peuvent émettre et recevoir
- Les modules composés :
 - Relais transparents
- Utilisation de **ports (gates)** :
 - Gates d'entrée et de sortie
 - Connexion entre un port de sortie et un port d'entrée
- Paramètres possibles des connexions :
 - Délai de propagation
 - Débit
 - Taux d'erreur

Les connexions ne sont autorisées qu'au **même niveau hiérarchique**

Installation de l'IDE OMNeT++ (Windows)

1. Ouvrir le répertoire OMNeT++
2. Lancer **Mingwenv.cmd**
3. Exécuter :
 - `./configure`
 - `make`
 - `omnetpp`
4. Créer un raccourci bureau vers `omnetpp.exe`
5. Ajouter `<répertoire_omnet>/bin` à la variable **PATH**

Remarques

- Compilateur **C++ obligatoire**
- Lien de téléchargement : <https://omnetpp.org/>

TP n°2 : Démarrer avec OMNeT++

Objectif du TP

- Apprendre à **créer un premier projet OMNeT++**
- Mettre en œuvre un **scénario simple Tic-Toc**
- Simuler l'échange de messages entre **deux nœuds** d'un réseau

Principe du scénario Tic-Toc

- Le réseau contient **deux nœuds : Tic et Toc**
- **Tic initie la communication** en envoyant un message
- À chaque réception, un nœud **renvoie un message** à l'autre
- L'échange se fait via des **messages OMNeT++**

Étapes de réalisation

1. Création de l'espace de travail

- Créer un dossier nommé **Omnetworkspace**
- L'utiliser comme **workspace** OMNeT++

2. Création du projet OMNeT++

- File → New → OMNeT++ Project
- Nom du projet : **tic-toc**
- Le projet apparaît dans l'explorateur avec son arborescence

3. Fichiers nécessaires au projet

Le projet repose sur **trois fichiers essentiels** :

1. **Fichier NED** → description de la topologie du réseau
2. **Classe C++** → comportement des nœuds
3. **Fichier INI** → configuration de la simulation

4. Création du fichier NED

- Créer un **Network Description File**
- Nom : **tictoc.ned**
- Contenu :
 - Définition d'un **module simple Noeud** avec une entrée et une sortie
 - Définition du réseau **Tictoc**
 - Deux sous-modules : **tic** et **toc**
 - Connexions bidirectionnelles avec un **délai de 100 ms**

5. Création de la classe C++ (Noeud)

- Créer une **classe OMNeT++**
- Nom : **Noeud**
- Fichiers générés :
 - **Noeud.h**
 - **Noeud.cc**

Fonctionnement

- **initialize()** :
 - Si le nœud est **tic**, il envoie le premier message
- **handleMessage()** :
 - À chaque message reçu, le nœud répond en envoyant un message

Les deux nœuds se renvoient les messages indéfiniment

6. Création du fichier INI

- Créer un **Initialization File**
- Nom : **omnet.ini**
- Rôle :
 - Définir le réseau à simuler (**Tictoc**)
 - Paramétrer la simulation (temps, options, etc.)

7. Compilation et simulation

- Compilation : **Project** → **Build All**
- Lancement :
 - Clic droit sur **omnet.ini**
 - Run As → **OMNeT++ Simulation**
- Résultat :
 - Une fenêtre graphique (animation du réseau)
 - Une fenêtre de logs et d'événements
- Démarrage : bouton **Run**

TP n°3 : Simulation d'un réseau sans fil

Objectif du TP

L'objectif de ce TP est de **simuler un réseau sans fil simple** sous OMNeT++ en utilisant le framework **INETMANET**, afin d'étudier la **communication entre deux nœuds sans fil** et d'analyser l'impact de la **portée radio** sur la connectivité.

I. Préparation de l'environnement

- Utilisation du framework **INETMANET-4.x**, une bibliothèque open source dédiée à la simulation des réseaux **sans fil et ad hoc**.
- Importation du framework INETMANET dans le **workspace OMNeT++**, suivie de la compilation du projet.

Création du scénario du TP dans le dossier : `inetmanet/examples/TP3`

- afin d'organiser les simulations et les fichiers NED/INI de manière structurée.

II. Description du réseau sans fil

Le réseau simulé est composé des éléments suivants :

1. Nœuds sans fil

- **hostA** et **hostB** : deux nœuds sans fil de type **WirelessHost**.
- Chaque nœud est capable :
 - d'émettre des paquets,
 - de recevoir des paquets,

- de communiquer via une interface radio IEEE 802.11.

2. Modules du réseau

- **radioMedium**
 - Représente le canal radio partagé.
 - Gère la portée de transmission, les interférences et la réception des signaux.
- **Configurator (Ipv4NetworkConfigurator)**
 - Attribue automatiquement les **adresses IP** aux nœuds.
 - Configure le routage IPv4.
- **GlobalArp**
 - Permet une **résolution ARP globale**.
 - Évite les requêtes ARP dynamiques.
 - Accélère la communication dans les scénarios simples.

Exemple : `*.host*.ipv4.arp.typename = "GlobalArp"`

- **Visualizer**
 - Assure l'affichage graphique :
 - des nœuds, des liens radio, des paquets échangés, des statistiques (paquets reçus).

III. Simulation de la communication

1. Applications réseau

- Utilisation de **UdpBasicApp** pour :
 - l'envoi de paquets UDP, la réception bidirectionnelle.
- Possibilité d'utiliser **UdpSink** lorsque l'un des nœuds joue uniquement le rôle de récepteur.

2. Modèle radio utilisé

- **Modèle unitDisk** :
 - Portée de transmission fixe.
 - Réception parfaite tant que le récepteur est à l'intérieur de la portée.
 - Aucune dégradation progressive du signal.

Interface sans fil : `*.host*.wlan[0].typename = "AckingWirelessInterface"`

- MAC simplifié.
- Communication principalement basée sur la distance entre les nœuds.

3. Paramètres importants

- Portée de transmission :
 - 500 m → communication directe possible.
 - 250 m → communication impossible si la distance est trop grande.
- Débit :

- 1 Mbps.
- Trafic :
 - Échange de paquets UDP entre hostA et hostB.

IV. Problème de portée et solutions

1. Augmentation de la puissance de transmission

Une première solution consiste à **augmenter la portée radio** :

```
*.host*.wlan[0].radio.transmitter.analogModel.communicationRange = 500m
```

Limites de cette solution :

- Augmente la consommation d'énergie.
- Accroît les interférences.
- Peu réaliste dans les réseaux de capteurs.
- **Ce n'est pas la meilleure solution** à grande échelle.

2. Solution recommandée : ajout de nœuds intermédiaires

La solution la plus réaliste consiste à **ajouter des nœuds intermédiaires** :

- Ces nœuds servent de **relais**.
- Les paquets sont transmis par **sauts multiples (multi-hop)**.
- La communication devient possible même avec une faible portée.

Exemple : hostA → hostR1 → hostR2 → hostB

Cette approche :

- respecte les contraintes énergétiques,
- améliore la scalabilité,
- prépare l'étude du routage (TP4).

TP n°4 : Configuration des modes de routage

Objectif du TP

Mettre en place, configurer et **comparer le routage statique et le routage dynamique** dans un réseau sans fil simulé sous **OMNeT++ / INETMANET**, afin d'analyser leur fonctionnement et leurs limites.

Partie A : Routage statique basé sur la métrique *errorRate*

Description du scénario

- Reprise du scénario du **TP3** avec deux nœuds sans fil :

- **hostA** et **hostB**
- Réduction de la portée radio :
 - de **500 m à 250 m**
 - ⇒ la communication directe devient impossible.
- Ajout de **nœuds intermédiaires (relais)** :
 - communication multi-sauts (multi-hop).

Problème

Même avec des nœuds relais, **les paquets ne sont pas transmis** :

- par défaut, un hôte **ne relaie pas** les paquets reçus,
- les tables de routage sont vides.

Solution : configuration du routage statique

1. Activation du forwarding

```
*.host*.forwarding = true
```

forwarding = true signifie que :

- chaque hôte peut agir comme **routeur**, il peut **retransmettre les paquets** vers d'autres nœuds, sans cela, aucun routage multi-sauts n'est possible.

2. Configuration automatique des routes statiques

Utilisation du module **Ipv4NetworkConfigurator** avec un fichier XML :

```
<config>
```

```
<interface hosts="*" address="10.0.0.x" netmask="255.255.255.0"/>
```

```
<autoroute metric="errorRate"/>
```

```
</config>
```

- Toutes les **adresses IP des hôtes** sont configurées automatiquement.
- La balise **<autoroute>** :
 - génère automatiquement les **routes statiques**,
 - utilise la métrique **errorRate**.

3. Signification des paramètres importants

- **metric = "errorRate"**
 - Le choix des routes se base sur le **taux d'erreur** des liens.
 - La route avec le **plus faible taux d'erreur** est privilégiée.
- **optimizeRoutes = false**

```
*.configurator.optimizeRoutes = false
```

Empêche le configurator de :

- supprimer des routes,
- ne garder qu'une seule route optimale.

Utile lorsqu'on souhaite **plusieurs chemins possibles** dans le réseau.

Observation des tables de routage (OMNeT++)

Pour visualiser les routes :

1. Cliquer sur un **host**
2. Aller dans :
IPv4 → Routing Table
3. En bas à gauche :
 - cliquer sur **Children**
 - puis sur **route**
4. Les routes apparaissent sous forme de **vecteur** contenant :
 - destination, **gw (gateway)**, interface utilisée.

Partie B : Tables de routage non configurées

Objectif

Préparer le réseau pour le **routage dynamique**.

Configuration

```
*.configurator.addStaticRoutes = false
```

Effet

- Le configurator :
 - attribue uniquement les **adresses IP**, ne crée aucune route statique.
- Les tables de routage restent **vides**.
- Sans protocole dynamique, la communication multi-sauts est impossible.

Partie C : Routage dynamique avec AODV

Mise en place d'AODV

- Utilisation du protocole **AODV (Ad-hoc On-demand Distance Vector)**.
- Désactivation complète du routage statique :

```
*.configurator.addStaticRoutes = false
```

- Transformation des hôtes en routeurs AODV :

```
*.host*.typename = "AodvRouter"
```


- Chemin :src/inet/node/aodv/

Fonctionnement d'AODV

- **Routage à la demande :**
 - une route est découverte uniquement lorsqu'un hôte veut communiquer.
- Mise à jour dynamique des routes.
- Adapté aux :
 - réseaux sans fil,réseaux mobiles,topologies changeantes.

Topologie du réseau

- **WirelessA :**
 - hostA et hostB.
- **WirelessB :**
 - ajout des relais : hostR1, hostR2, hostR3.
- Support radio :
 - **RadioMedium**
 - modèle **unitDisk**.
- Interface :
 - **AckingWirelessInterface**
- Débit :
 - **1 Mbps**.

Limites du routage statique

- Nécessite une **connaissance préalable** de la topologie.
- Peu adapté aux réseaux :
 - mobiles,dynamiques.
- Toute modification du réseau nécessite une **reconfiguration**.

D'où l'intérêt du **routage dynamique**.

Pourquoi le nombre de paquets reçus est le même (statique vs dynamique) ?

- La topologie est **fixe**.
- Les nœuds ne bougent pas.
- Les routes (statique ou AODV) mènent au **même chemin final**.
- Le trafic UDP est identique.

Résultat :

- **Même chemin**
- **Même débit**
- **Même nombre de paquets reçus**

La différence entre statique et dynamique apparaît surtout lorsque :

- les nœuds se déplacent,des liens se coupent,la topologie change

TP n°5 – Gestion de la mobilité

Objectif du TP

Jusqu'ici, les nœuds étaient **statiques** (positions fixées dans le fichier `.ned`).

Ce TP introduit la **mobilité des nœuds** afin de simuler des **réseaux sans fil réalistes**, où les nœuds peuvent :

- rester immobiles, se déplacer selon des trajectoires définies, changer de position au cours du temps.

Pour cela, on utilise le **module mobility** intégré aux nœuds.

Principe général

- Les **positions ne sont plus définies dans le fichier .ned**
- Elles sont définies dans le **fichier omnetpp.ini**
- Chaque nœud possède un module `:hostX.mobility`

Partie 1 : Placement stationnaire (StationaryMobility)

Principe

Les nœuds **ne bougent pas**, mais leurs positions sont :

- configurées dynamiquement
- modifiables sans toucher au fichier `.ned`

Le modèle utilisé est `:StationaryMobility`

- Chemin `:src/inet/mobility/static`

Paramètres principaux

- `initialX`, `initialY`, `initialZ` : position du nœud
- `initFromDisplayString` :
 - **false** → on force la position via le `.ini`
 - **true** → position prise depuis l'interface graphique

Étape 1 : Zone de déploiement

Définition de la zone où les nœuds peuvent être placés :

```
**constraintAreaMinX = 0m
```

```
**constraintAreaMinY = 0m
```

```
**constraintAreaMaxX = 1000m
```

```
**constraintAreaMaxY = 1000m
```

Tous les nœuds seront placés dans ce carré 1000m × 1000m.

Étape 2 : Exemple de placement fixe

```
*.hostA.mobility.typename = "StationaryMobility"
```

```
*.hostA.mobility.initialX = 50m
```

```
*.hostA.mobility.initialY = 325m
```

```
*.hostA.mobility.initFromDisplayString = false
```

hostA est **immobile** et placé précisément à (50, 325).

Même logique pour les relais :

```
*.hostR1.mobility.initialX = 250m
```

```
*.hostR1.mobility.initialY = 300m
```

Exemple concret

- hostA et hostB sont hors de portée directe
- hostR1, hostR2, hostR3 servent de **relais fixes**
- Le routage (statique ou AODV) fonctionne **sans déplacement**

Utilité

- Tester le routage
- Reproduire exactement une topologie donnée
- Comparer avec les TPs précédents

Partie 2 : Mobilité avec fichier XML (AnsimMobility)

- Chemin :src/inet/mobility/single

Problème

Quand il y a beaucoup de nœuds :

- définir chaque position manuellement devient lourd
- impossible de gérer des déplacements complexes facilement

Solution

Utiliser le modèle :AnsimMobility

Il lit les positions (et déplacements) depuis un **fichier XML**.

Étape 1 : Tableau de nœuds

Dans le .ned :

host[numHosts]: WirelessHost

Chaque nœud a un **nodeId** :host[0], host[1], host[2], ...

Étape 2 : Exemple XML (déplacement)

```
<position_change>
  <node_id>0</node_id>
  <start_time>0</start_time>
  <end_time>5</end_time>
  <destination>
    <xpos>50</xpos>
    <ypos>50</ypos>
  </destination>
</position_change>
```

Le nœud 0 se déplace vers (50,50) entre 0s et 5s.

Configuration INI

```
** .host[*].mobility.typename = "AnsimMobility"
** .host[*].mobility.ansimTrace = xmldoc("ansimtrace.xml")
** .host[*].mobility.nodeId = -1
```

nodeId = -1 signifie :

l'ID du nœud = son index dans le tableau

Exemple concret

- Les nœuds démarrent tous au même point
- Puis se déplacent vers leurs positions finales
- Les liens apparaissent / disparaissent selon la distance
- AODV met à jour les routes automatiquement

Partie 3 : Autres modèles de mobilité

Le framework **INETMANET** fournit plusieurs modèles prêts à l'emploi.

1. LinearMobility

Déplacement en ligne droite.

```
** .host[*].mobility.typename = "LinearMobility"  
  
** .host[*].mobility.speed = 50mps  
  
** .host[*].mobility.initialMovementHeading = 30deg
```

Exemple : véhicules roulant sur une route.

2. CircleMobility

Déplacement circulaire autour d'un point.

```
** .host[*].mobility.typename = "CircleMobility"  
  
** .host[*].mobility.r = 150m  
  
** .host[*].mobility.speed = 40mps
```

Exemple : drones ou capteurs tournant autour d'une zone.

3. RandomWaypointMobility

- Destination aléatoire
- Vitesse aléatoire
- Déplacements par segments

Très utilisé pour simuler des réseaux ad hoc.

4. RectangleMobility

Déplacement le long des bords d'un rectangle.

5. TractorMobility

Simulation réaliste d'un tracteur se déplaçant en rangées.

TP n°6 – Gestion de la consommation énergétique

Objectif du TP

Dans les **réseaux de capteurs sans fil**, l'énergie est une ressource **limitée** car les nœuds sont alimentés par des **batteries**.

L'objectif de ce TP est de :

- **modéliser la consommation énergétique** des capteurs, **suivre l'énergie consommée ou restante**, **gérer l'arrêt et le redémarrage des nœuds** lorsque la batterie se vide ou se recharge.

Principe général

La consommation d'énergie dans INET / InetManet repose sur **trois composants principaux** :

1. **Energy Consumer** → qui consomme l'énergie
2. **Energy Storage** → qui stocke l'énergie (batterie)
3. **Energy Management** → qui décide quand arrêter ou redémarrer un nœud

1. Modélisation de la consommation d'énergie

1.1 Source de consommation

Dans un capteur sans fil, l'énergie est consommée par :

- **l'émission** des messages (très coûteuse), la **réception** des messages, **l'écoute du canal radio**, les états **veille (sleep)** ou **arrêt (off)**.

Même **sans transmettre**, un capteur consomme de l'énergie.

1.2 Modèle utilisé : **StateBasedEpEnergyConsumer**

- Chemin: `src/inet/physicallayer/wireless/common/energyconsumer/StateBasedEpEnergyConsumer`

Principe

Ce modèle calcule la consommation selon :

- l'état de la radio, des **paramètres constants** de puissance.

États de la radio et consommation (exemples)

État de la radio	Consommation
------------------	--------------

Off	0 mW
Sleep	1 mW
Réception	10 mW
Émission	100 mW

L'émission consomme beaucoup plus que la réception

Plus la distance est grande, plus l'émission est sollicitée (couche physique).

Exemple de configuration INI

```
**energyConsumer.typename = "StateBasedEpEnergyConsumer"
```

2. Modélisation du stockage d'énergie (batterie)

Chaque nœud possède une **batterie**, modélisée par **energyStorage**.

Deux modèles sont utilisés dans ce TP.

2.1 IdealEpEnergyStorage (énergie infinie)

- Chemin :src/inet/power/storage/IdealEpEnergyStorage

Caractéristiques

- Énergie illimitée
- Le nœud **ne s'éteint jamais**
- Sert à **observer la consommation**, pas l'épuisement

Variable importante

energyBalance : représente l'énergie consommée cumulée

Exemple : afficher l'énergie consommée

```
std::string node_name = this->getParentModule()->getFullName();
```

```
cout << "Je suis le noeud " << node_name
```

```
<< " J'ai consommé " << energyBalance << endl;
```

- getParentModule() → récupérer le nœud
- getFullName() → nom du capteur

Configuration INI

```
**energyStorage.typename = "IdealEpEnergyStorage"
```

Utilisé pour **observer le comportement énergétique sans coupure**

2.2 SimpleEpEnergyStorage (batterie réelle)

- Chemin :src/inet/power/storage/SimpleEpEnergyStorage

Caractéristiques

- Batterie **limitée**
- Énergie diminue avec le temps
- Le nœud peut **s'éteindre**

Variable importante

residualCapacity:énergie restante du nœud

Exemple de paramètres

```
**energyStorage.typename = "SimpleEpEnergyStorage"
```

```
**energyStorage.nominalCapacity = 0.5J
```

La batterie contient **0.5 joule maximum**

Exemple d'affichage

```
cout << "Je suis le noeud " << node_name
```

```
<< " Mon energie restante est : "
```

```
<< residualCapacity << endl;
```

3. Gestion automatique de l'énergie

- Chemin :src/inet/power/management/SimpleEpEnergyManagement

Modèle utilisé : SimpleEpEnergyManagement

Rôle

- Surveille l'énergie résiduelle
- **Éteint le nœud** si l'énergie est trop faible
- **Redémarre le nœud** si l'énergie remonte

Exemple de configuration

```
**energyManagement.typename = "SimpleEpEnergyManagement"
```

```
**energyManagement.nodeShutdownCapacity = 0J
```

```
**energyManagement.nodeStartCapacity = 0.025J
```

Scénario concret

1. Le capteur transmet des données UDP
2. La batterie se vide progressivement
3. Quand `residualCapacity = 0J` :
 - le nœud est **arrêté automatiquement**
4. Si l'énergie remonte à `0.025J` :
 - le nœud **redémarre**

4. Lien avec les couches réseau

- La communication se fait via des **ondes radio**
- La consommation dépend de la **couche physique**
- La mobilité + routage + énergie = réseau réaliste

Un capteur peut :

- transmettre → forte consommation
- recevoir → consommation moyenne
- écouter → consommation faible
- dormir → très faible consommation

TP n°7 – Création d'une application de capture de température

Objectif du TP

L'objectif de ce TP est de **concevoir une application réseau personnalisée** dans OMNeT++/INET permettant de :

- simuler des **nœuds capteurs** qui mesurent une **température**,
- envoyer ces données vers un **nœud de collecte (sink)**,
- **agrégier les données** côté sink,
- calculer et afficher la **température moyenne** reçue.

Le réseau est composé de :

- plusieurs nœuds `host[i]` (capteurs),
- un seul nœud `sink` (station de base).

Architecture générale

Deux applications créées

1. `ApplicationNodeTempSensing`

- exécutée sur chaque nœud capteur, capture une valeur de température, l'envoie périodiquement au sink via UDP.

2. `ApplicationSink`

- exécutée sur le nœud sink, reçoit toutes les données, stocke les valeurs reçues, calcule la **moyenne des températures**.

1. Modification du message `ApplicationPacket`

Objectif

Pouvoir transporter :

- la température, l'identifiant du nœud source, le type de paquet.

Fichier modifié

`inet/src/inet/applications/base/ApplicationPacket.msg`

Champs ajoutés

```
uint32_t sequenceNumber;
```

```
double data;      // température
```

```
int src;          // ID du nœud source
```

```
string PacketType; // DATA
```

Rôle

- `data` → valeur de température envoyée
- `src` → identifier quel nœud a envoyé la donnée
- `PacketType` → distinguer les types de messages

2. ApplicationNodeTempSensing (nœuds capteurs)

Principe

Chaque nœud :

1. génère une valeur de température,
2. encapsule cette valeur dans `ApplicationPacket`,
3. envoie le paquet au sink.

Fonction clé : `sendPacket()`

Exemple de logique

```
payload->setData(rand() % 100);  
  
payload->setSrc(this->getParentModule()->getIndex());  
  
payload->setPacketType("DATA");
```

Explication

- `rand()%100` → température simulée (0–99)
- `getIndex()` → ID du nœud dans le vecteur `host[]`
- encapsulation avec `insertAtBack(payload)`

Gestion de l'énergie

```
double energy_node = this->getParentModule()->par("energy");  
  
if (energy_node != 0) {  
    // envoyer le paquet  
}
```

Un nœud n'envoie des données que s'il a encore de l'énergie.

Envoi UDP

```
socket.sendTo(packet, destAddr, destPort);  
  
• destAddr = "sink"  
• destPort = 1234
```

3. ApplicationSink (nœud de collecte)

Principe

Le sink :

- reçoit les paquets, extrait les données, stocke les valeurs, calcule la moyenne lorsque tous les nœuds ont envoyé.

Structure de stockage

Définie dans ApplicationSink.h :

```
struct enregistrement {
    int neighborID;
    double data;
    string packetType;
};

list<enregistrement> Tab;
```

Chaque entrée correspond à **une mesure reçue**.

Fonction clé : processPacket()

Décapsulation du paquet

```
const auto& appPacket = pk->peekAtFront<ApplicationPacket>();
```

peekAtFront permet **d'accéder au contenu sans modifier le paquet**.

Enregistrement des données

```
E.neighborID = appPacket->getSrc();
```

```
E.data = appPacket->getData();
```

```
Tab.insert(Tab.end(), E);
```

Calcul de la moyenne

```
if (Tab.size() == numHosts) {
    for (it = Tab.begin(); it != Tab.end(); it++)
        S += (*it).data;
    Moy = S / Tab.size();
}
```

Résultat affiché

- somme des températures,
- moyenne finale,
- identification du sink.

4. Configuration du réseau (omnet.ini)

Topologie

- numHosts = 4
- 1 sink placé au centre
- nœuds fixes (StationaryMobility)

Applications

```
*.sink.app[0].typename = "ApplicationSink"
```

```
** .host[*].app[0].typename = "ApplicationNodeTempSensing"
```

```
** .host[*].app[0].destAddresses = "sink"
```

Envoi périodique

```
** .host[*].app[0].sendInterval = exponential(10s)
```

Évite les collisions et rend la simulation réaliste.

5. Gestion de l'énergie (lien avec TP6)

Nœuds capteurs

```
** .host[*].energyStorage.typename = "SimpleEpEnergyStorage"
```

```
** .host*.energyStorage.nominalCapacity = 1.5J
```

Batterie limitée → nœuds peuvent s'éteindre.

Sink

```
** .sink.energyStorage.typename = "IdealEpEnergyStorage"
```

Énergie infinie → toujours actif.

6. Résumé

- Les capteurs :
 - mesurent une température,
 - envoient des données UDP au sink

- Le sink :reçoit,stocke,calcule la moyenne
- Utilisation de :
 - ApplicationPacket modifié,sendPacket() pour l'envoi,processPacket() pour la réception
- Intégration de la **consommation énergétique**

MAGUEMOUN SAMY